DATA SCIENCE SUBJECT DIGITAL LIBRARY ? STUDY GUIDE

Module 3
Data Visualization

Hands-on visualization techniques using Python libraries to represent patterns, comparisons, distributions, and relationships clearly and effectively.

8 teaching periods ? 8 topics ? Learning objectives and full topic notes

Learning Objectives

- Use the matplotlib API to create figures, axes, legends, titles, labels, and multiple subplots.
- Create line plots to display trends and temporal changes in datasets.
- Construct bar plots, grouped bar charts, and categorical comparisons using Pandas and Seaborn.
- Generate histograms and kernel density plots to study the shape and spread of numerical data.
- Use scatter plots to explore relationships between two or more variables.
- Create facet grids to compare data across multiple categories or groups.
- Choose appropriate chart types for different kinds of analytical questions.
- Explore additional Python visualization libraries such as Plotly, Bokeh, and Altair.

Contents

1. Matplotlib API Primer: Figures, Axes, Subplots, and Customization
2. Line Plots with Pandas and Seaborn
3. Bar Plots, Grouped Bars, and Categorical Comparisons
4. Histograms and Density Plots
5. Scatter Plots and Bubble Charts
6. Facet Grids and Categorical Data Visualization
7. Choosing Suitable Visualizations for Data Storytelling
8. Other Python Visualization Tools: Plotly, Bokeh, and Altair

Topic Notes

TOPIC 01

Matplotlib API Primer: Figures, Axes, Subplots, and Customization

Matplotlib is the foundational Python visualization library built around two key objects: the Figure (the entire canvas) and Axes (an individual plot area within the figure). A Figure can contain multiple Axes objects arranged in a grid using plt.subplots(nrows, ncols). Each Axes supports methods like ax.plot(), ax.bar(), ax.set_title(), ax.set_xlabel(), ax.legend(), and ax.grid(). Customization options include colors, line styles, markers, font sizes, tick formatting, and spine visibility. This object hierarchy allows precise control over every visual element, unlike the simpler but less flexible pyplot functional interface.

TOPIC 02

Line Plots with Pandas and Seaborn

Line plots are the primary tool for visualizing data that changes continuously over an ordered axis â€” most commonly time. In Pandas, df.plot(kind='line') automatically uses the index as the x-axis, making temporal datasets easy to visualize. Seaborn's sns.lineplot() adds confidence intervals around the mean by default when multiple observations exist per x value, making it powerful for visualizing trends in grouped data. Best practices include clearly labeling axes with units, using different colors/dashes for multiple series, and starting the y-axis at a meaningful baseline to avoid misleading visual exaggeration.

TOPIC 03

Bar Plots, Grouped Bars, and Categorical Comparisons

Bar plots compare discrete categories along one axis and a quantitative measure along the other. A simple bar chart compares totals or averages across a single categorical variable. Grouped bar charts place bars for different sub-categories side by side, enabling comparison across two categorical dimensions. Stacked bar charts show composition â€” each bar is divided into segments representing sub-categories. Seaborn's sns.barplot() automatically computes confidence intervals, while sns.countplot() counts occurrences directly. Best practices: sort bars by value for readability, use horizontal bars when labels are long, and avoid 3D bars which distort perception.

TOPIC 04

Histograms and Density Plots

Histograms display the distribution of a single numerical variable by dividing the range into bins and counting how many data points fall in each bin. The choice of bin width significantly affects interpretation: too few bins obscure detail, too many create noise. Kernel Density Estimation (KDE) plots smooth the histogram into a continuous probability density curve by fitting a Gaussian kernel at each data point and summing them. Seaborn's sns.histplot(kde=True) overlays both. Density plots are particularly useful for comparing distributions of two or more groups on the same axes, and for identifying skewness, multimodality, and outliers.

TOPIC 05

Scatter Plots and Bubble Charts

Scatter plots visualize the relationship between two continuous variables by plotting each observation as a point with coordinates (x, y). They are the primary tool for identifying correlation, clusters, and outliers. When a third variable is encoded as the size of each point, the chart becomes a bubble chart â€” useful for comparing three dimensions simultaneously (e.g., GDP vs. life expectancy with population as circle size). Seaborn's sns.scatterplot() supports hue (color grouping) and size aesthetics directly. Adding a regression line with sns.regplot() visualizes the trend, while residual plots check linearity assumptions.

TOPIC 06

Facet Grids and Categorical Data Visualization

Facet grids (small multiples) create an array of subplots, each showing a subset of the data defined by one or more categorical variables. Seaborn's FacetGrid class enables this: g = sns.FacetGrid(df, col='category_A', row='category_B'), followed by g.map(plt.hist, 'numerical_column'). This technique is powerful for comparing distributions, trends, or relationships across different groups simultaneously â€” for example, showing income distribution across 5 different cities in a 1x5 grid. Facet grids prevent overloaded single charts and make patterns that vary by subgroup immediately visible.

TOPIC 07

Choosing Suitable Visualizations for Data Storytelling

Choosing the right chart type is a critical analytical skill. Use line charts for time series. Use bar charts for comparing discrete categories. Use histograms and KDE plots for distributions. Use scatter plots for correlations. Use heatmaps for correlation matrices. Use box plots for comparing distributional spread across groups. Data storytelling layers appropriate charts with a clear narrative: starting with the big picture (overall trend), zooming into interesting patterns (anomalies), and closing with a clear takeaway. Key principles: maximize the data-ink ratio (remove chart junk), use color purposefully not decoratively, and always label clearly.

TOPIC 08

Other Python Visualization Tools: Plotly, Bokeh, and Altair

Beyond Matplotlib and Seaborn, the Python ecosystem offers powerful interactive and declarative visualization libraries. Plotly creates interactive charts (hover tooltips, zoom, pan) that work natively in Jupyter notebooks and web browsers; Plotly Express offers a high-level interface similar to Seaborn. Bokeh is designed for building interactive dashboards and web applications, with server-side callbacks enabling real-time data updates. Altair uses a declarative, grammar-of-graphics approach (based on Vega-Lite) where charts are described as specifications mapping data columns to visual channels (x, y, color, size). Each library has trade-offs: Plotly for quick interactivity, Bokeh for dashboards, and Altair for elegant statistical graphics.

Module notes: A good visualization should reveal the underlying pattern in the data without hiding important uncertainty or distortion. Matplotlib provides detailed control over every chart element, while Seaborn offers concise statistical plots. Facet grids allow multiple plots to be compared under different categories, which makes it easier to identify trends, clusters, and outliers.